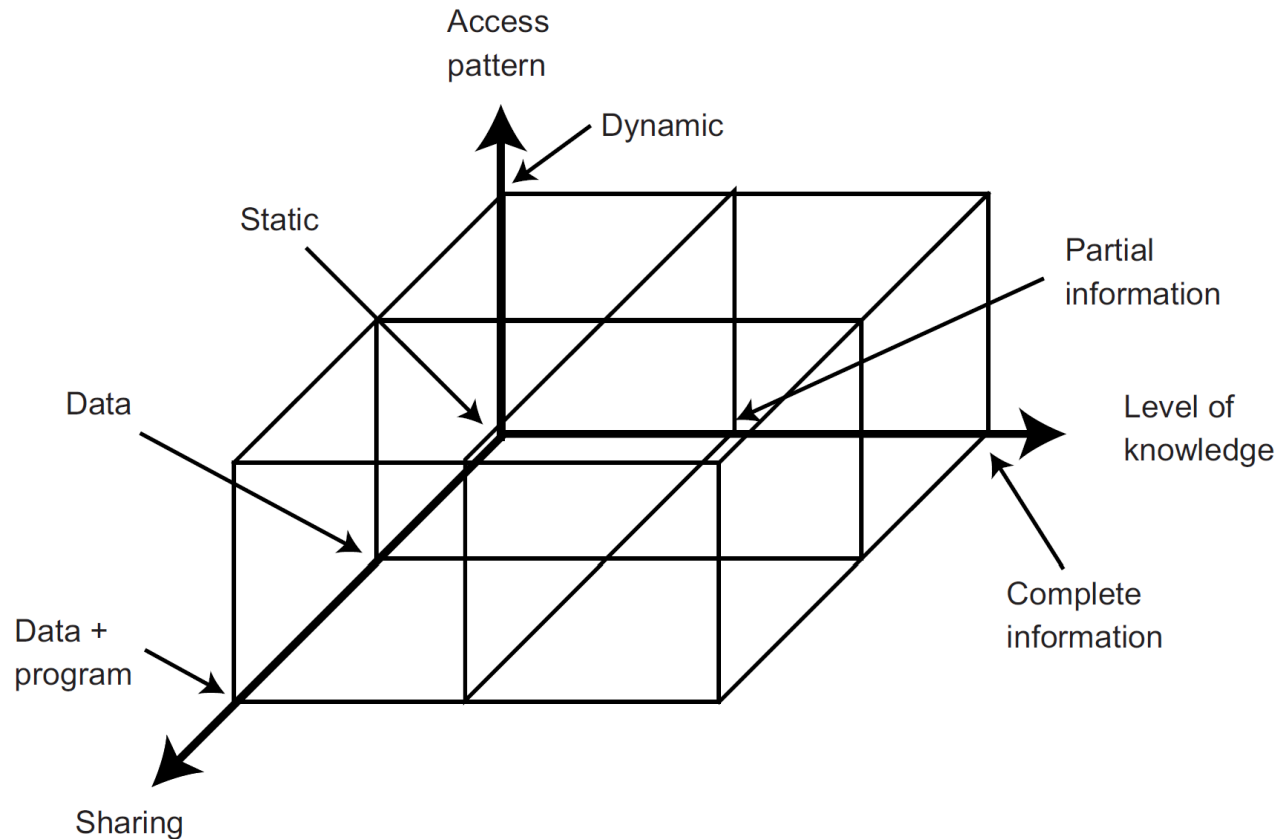

Principles of Distributed Database Systems

M. Tamer Özsu
Patrick Valduriez

Outline

- Distributed and Parallel Database Design
 - ▣ Fragmentation
 - ▣ Data distribution
 - ▣ Combined approaches

Distributed database design



Framework of Distribution

Distributed database design

Dimensions of Distributed System Organization

Three Orthogonal Dimensions (Levin & Morgan, 1975):

1. Level of Sharing

1. **No Sharing:** Program + data at one site (rare today).
2. **Data Sharing:** Programs replicated at all sites; only data moves.
3. **Data + Program Sharing:** Programs + data shared across sites (common in heterogeneous systems).

2. Behavior of Access Patterns

1. **Static:** Access predictable, easy to manage (rare in real life).
2. **Dynamic:** Access varies over time → harder to manage.

3. Level of Knowledge on Access Patterns

1. **No Information:** Impractical for design.
2. **Complete Information:** Predictable and stable.
3. **Partial Information:** Predictions exist but with deviations.

Distributed Database Design Strategies

Two Design Approaches:

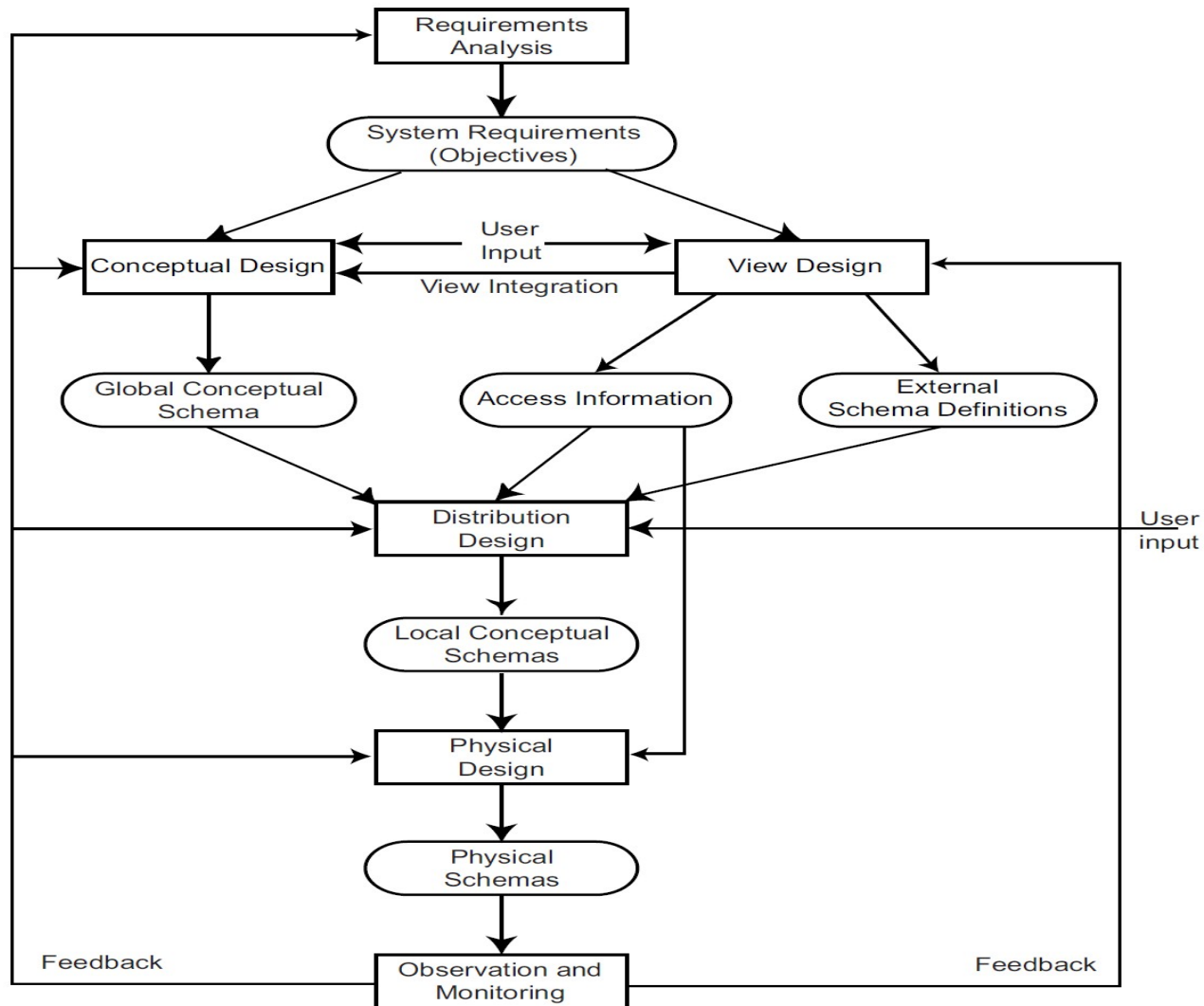
Top-Down Approach

- Suitable for **homogeneous, tightly integrated DBMS**.
- Start with global schema → decompose into fragments → allocate to sites.

Bottom-Up Approach

- Suitable for **heterogeneous, multidatabase systems**.
- Start with existing local schemas → integrate into global schema.

Top-Down Design Process



Top-Down Design Process

Steps in the Process (Figure):

1. Requirements Analysis

Defines environment, user needs (data + processing).

Objectives: **Performance, Reliability, Availability, Economics, Expandability.**

2. View Design

Defines user interfaces & application views.

Collects access pattern & statistical info (frequency, volume, usage).

3. Conceptual Design

Entity Analysis: Identify entities, attributes, relationships.

Functional Analysis: Identify enterprise functions & link to entities.

Integration of user views into a **Global Conceptual Schema (GCS).**

Top-Down Design Process

4. Distribution Design

Inputs: GCS + access patterns.

Goal: Create **Local Conceptual Schemas (LCSs)**.

Two steps:

Fragmentation → Divide relations into sub-relations (fragments).

Allocation → Assign fragments to sites.

5. Physical Design

Map LCSs to physical storage at sites.

Optimize using access patterns.

6. Observation & Monitoring

Continuous evaluation of:

Database performance.

Suitability of user views.

Provides feedback → may require revisiting earlier design steps.

Top-Down Design Issues – Key Questions

Main Questions in Distribution Design

1. Why fragment at all?
2. How should we fragment?
3. How much should we fragment?
4. How to test correctness of decomposition?
5. How should we allocate?
6. What information is needed for fragmentation & allocation?

Fragmentation – Reasons & Alternatives

1. Why Fragment?

- Application views are subsets of relations → improves **locality of access**.
- Avoids excessive replication & remote accesses.
- Improves **concurrency & throughput** via parallel query execution.
- Challenges: costly distributed joins, integrity checking issues.

2. How should we fragment? - Fragmentation Alternatives

1. **Horizontal Fragmentation** → divide tuples into subsets.
2. **Vertical Fragmentation** → divide attributes into subsets.
3. **Hybrid Fragmentation** → nesting of horizontal + vertical.

Distribution Design Issues – Key Questions

3. How much should we fragment? - **Degree of Fragmentation**

Range: **No fragmentation** → **per tuple/attribute**.

Trade-off between too coarse (poor locality) vs too fine (high overhead).

4. How to test correctness of decomposition? - **Correctness Rules**

Completeness → No loss of data.

Reconstruction → Global relation recoverable from fragments.

Disjointness → Horizontal fragments non-overlapping; vertical = PK repeated.

5. How should we allocate?- **Allocation Alternatives**

Partitioned DB → One copy per fragment.

Fully Replicated → Entire DB at every site.

Partially Replicated → Fragments replicated at selected sites.

6. What information is needed for fragmentation & allocation?-**Information Needed for Design**

Database Information (schema, dependencies).

Application Information (access patterns, query stats).

Network Information (bandwidth, latency).

System Information (CPU, storage, memory).

Outline

- Distributed and Parallel Database Design
 - ▣ Fragmentation
 - ▣ Data distribution
 - ▣ Combined approaches

Fragmentation

- Can't we just distribute relations?
- What is a reasonable unit of distribution?
 - relation
 - views are subsets of relations → locality
 - extra communication
 - fragments of relations (sub-relations)
 - concurrent execution of a number of transactions that access different portions of a relation
 - views that cannot be defined on a single fragment will require extra processing
 - semantic data control (especially integrity enforcement) more difficult

Example Database

EMP

<u>ENO</u>	ENAME	TITLE
E1	J. Doe	Elect. Eng.
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ASG

<u>ENO</u>	<u>PNO</u>	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E8	P3	Manager	40

PROJ

<u>PNO</u>	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris

PAY

<u>TITLE</u>	SAL
Elect. Eng.	40000
Syst. Anal.	34000
Mech. Eng.	27000
Programmer	24000

Correctness of Fragmentation

■ Completeness

- Decomposition of relation R into fragments $R_1, R_2, \dots, R_n$ is complete if and only if each data item in R can also be found in some R_i

■ Reconstruction

- If relation R is decomposed into fragments $R_1, R_2, \dots, R_n$, then there should exist some relational operator ∇ such that

$$R = \nabla_{1 \leq i \leq n} R_i$$

■ Disjointness

- If relation R is decomposed into fragments $R_1, R_2, \dots, R_n$, and data item d_i is in R_j , then d_i should not be in any other fragment R_k ($k \neq j$).

Fragmentation Alternatives

- Horizontal Fragmentation (HF)
 - ▣ Primary Horizontal Fragmentation (PHF)
 - ▣ Derived Horizontal Fragmentation (DHF)
- Vertical Fragmentation (VF)
- Hybrid Fragmentation (HF)

Fragmentation Alternatives – Horizontal

- Relation divided into **subsets of tuples (rows)** based on a condition (predicate).
- Each fragment has **same attributes** as the original relation.

Why Horizontal Fragmentation?

- Matches **user/application views** that access subsets of data.
- **Reduces** remote data access.
- Enables **parallel execution** of queries across fragments.

Fragmentation Alternatives – Horizontal

Example (PROJ Relation):

- Original **PROJ** relation includes project details.
- Fragmentation by **Budget**:
 - **PROJ₁** → Projects with Budget < 200,000
 - **PROJ₂** → Projects with Budget ≥ 200,000

PROJ₁ : projects with budgets less than \$200,000

PROJ₂ : projects with budgets greater than or equal to \$200,000

PROJ

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris

PROJ₁

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York

PROJ₂

PNO	PNAME	BUDGET	LOC
P3	CAD/CAM	255000	New York
P4	Maintenance	310000	Paris

Fragmentation Alternatives – Horizontal

Horizontal Fragmentation – Types

Two Types:

■ Primary Horizontal Fragmentation (PHF)

- Based on **selection predicates** defined on **the relation itself**.
- Tuples are partitioned according to conditions on their own attributes.

■ Derived Horizontal Fragmentation (DHF)

- Based on a relation that is **referenced by another relation**.
- Tuples are partitioned according to a join condition with another fragmented relation.

Fragmentation Alternatives – Horizontal

Primary Horizontal Fragmentation (PHF)

Definition

- Relation is fragmented directly using **predicates on its attributes**.

Example (PROJ Relation):

- **PROJ1** → Projects with Budget < 200,000
- **PROJ2** → Projects with Budget ≥ 200,000

Properties:

- Simple, direct partitioning.
- Useful when applications often access subsets defined by these predicates.

PHF - Information Requirements

■ Application Information

- **simple predicates** : Given $R[A_1, A_2, \dots, A_n]$, a simple predicate p_j is

$$p_j : A_i \theta Value$$

where $\theta \in \{=, <, \leq, >, \geq, \neq\}$, $Value \in D_i$ and D_i is the domain of A_i .

For relation R we define $Pr = \{p_1, p_2, \dots, p_m\}$

Example :

PNAME = "Maintenance"

BUDGET $\leq$ 200000

- **minterm predicates** : Given R and $Pr = \{p_1, p_2, \dots, p_m\}$

define $M = \{m_1, m_2, \dots, m_r\}$ as

$$M = \{ m_i \mid m_i = \bigwedge_{p_j \in Pr} p_j^* \}, 1 \leq j \leq m, 1 \leq i \leq r$$

where $p_j^* = p_j$ or $p_j^* = \neg(p_j)$.

PHF – Information Requirements

Example

m_1 : PNAME="Maintenance" $\wedge$ BUDGET $\leq$ 200000

m_2 : **NOT**(PNAME="Maintenance") $\wedge$ BUDGET $\leq$ 200000

m_3 : PNAME= "Maintenance" $\wedge$ **NOT**(BUDGET $\leq$ 200000)

m_4 : **NOT**(PNAME="Maintenance") $\wedge$ **NOT**(BUDGET $\leq$ 200000)

PHF – Information Requirements

■ Application Information

□ **minterm selectivities:** $sel(m_i)$

- The number of tuples of the relation that would be accessed by a user query which is specified according to a given minterm predicate m_i .

□ **access frequencies:** $acc(q_i)$

- The frequency with which a user application q_i accesses database.
- Access frequency for a minterm predicate can also be defined.

Primary Horizontal Fragmentation

Definition :

$$R_j = \sigma_{F_j}(R), \quad 1 \leq j \leq w$$

where F_j is a selection formula, which is (preferably) a minterm predicate.

Therefore,

A horizontal fragment R_j of relation R consists of all the tuples of R which satisfy a minterm predicate m_j .



Given a set of minterm predicates M , there are as many horizontal fragments of relation R as there are minterm predicates.

Set of horizontal fragments also referred to as **minterm fragments**.

PHF – Algorithm

Given: A relation R , the set of simple predicates Pr

Output: The set of fragments of $R = \{R_1, R_2, \dots, R_w\}$ which obey the fragmentation rules.

Preliminaries :

- Pr should be *complete*
- Pr should be *minimal*

Completeness of Simple Predicates

- A set of simple predicates Pr is said to be *complete* if and only if the accesses to the tuples of the minterm fragments defined on Pr requires that two tuples of the same minterm fragment have the same probability of being accessed by any application.

- Example :
 - ❑ Assume PROJ[PNO,PNAME,BUDGET,LOC] has two applications defined on it.
 - ❑ Find the budgets of projects at each location. (1)
 - ❑ Find projects with budgets less than \$200000. (2)

Completeness of Simple Predicates

According to (1),

$$Pr = \{LOC = \text{"Montreal"}, LOC = \text{"New York"}, LOC = \text{"Paris"}\}$$

which is not complete with respect to (2).

Modify

$$Pr = \{LOC = \text{"Montreal"}, LOC = \text{"New York"}, LOC = \text{"Paris"}, \\ BUDGET \leq 200000, BUDGET > 200000\}$$

which is complete.

Minimality of Simple Predicates

- If a predicate influences how fragmentation is performed, (i.e., causes a fragment f to be further fragmented into, say, f_i and f_j) then there should be **at least one application** that accesses f_i and f_j differently.
- In other words, the simple predicate should be **relevant** in determining a fragmentation.
- If all the predicates of a set Pr are relevant, then Pr is *minimal*.

$$\text{acc}(m_i) / \text{card}(f_i) \neq \text{acc}(m_j) / \text{card}(f_j)$$

Minimality of Simple Predicates

Example :

$$Pr = \{ \text{LOC} = \text{"Montreal"}, \text{LOC} = \text{"New York"}, \text{LOC} = \text{"Paris"}, \\ \text{BUDGET} \leq 200000, \text{BUDGET} > 200000 \}$$

is minimal (in addition to being complete). However, if we add

$$\text{PNAME} = \text{"Instrumentation"}$$

then Pr is not minimal.

COM_MIN Algorithm

Given: a relation R and a set of simple predicates Pr

Output: a *complete* and *minimal* set of simple predicates Pr' for Pr

Rule 1: a relation or fragment is partitioned into **at least two parts** which are **accessed differently** by at least one application.

COM_MIN Algorithm

1 Initialization :

- ❑ find a $p_i \in Pr$ such that p_i partitions R according to **Rule 1**
- ❑ set $Pr' = p_i$; $Pr \leftarrow Pr - \{p_i\}$; $F \leftarrow \{f_i\}$

2 Iteratively add predicates to Pr' until it is complete

- ❑ find a $p_j \in Pr$ such that p_j partitions some f_j defined according to minterm predicate over Pr' according to *Rule 1*
- ❑ set $Pr' = Pr' \cup \{p_j\}$; $Pr \leftarrow Pr - \{p_j\}$; $F \leftarrow F \cup \{f_j\}$
- ❑ if $\exists p_k \in Pr'$ which is nonrelevant then

$$Pr' \leftarrow Pr' - \{p_k\}$$

$$F \leftarrow F - \{f_k\}$$

PHORIZONTAL Algorithm

Makes use of COM_MIN to perform fragmentation.

Input: a relation R and a set of simple predicates Pr

Output: a set of minterm predicates M according to which relation R is to be fragmented

- ① $Pr' \leftarrow \text{COM_MIN}(R, Pr)$
- ② determine the set M of minterm predicates
- ③ determine the set I of implications among $p_i \in Pr$
- ④ eliminate the contradictory minterms from M

PHF – Example

- Two candidate relations : PAY and PROJ.
- Fragmentation of relation PAY
 - ❑ Application: Check the salary info and determine raise.
 - ❑ Employee records kept at two sites $\Rightarrow$ application run at two sites
 - ❑ Simple predicates
$$p_1 : \text{SAL} \leq 30000$$
$$p_2 : \text{SAL} > 30000$$
$$Pr = \{p_1, p_2\} \text{ which is complete and minimal } Pr' = Pr$$
 - ❑ Minterm predicates
$$m_1 : (\text{SAL} \leq 30000)$$
$$m_2 : \text{NOT}(\text{SAL} \leq 30000) = (\text{SAL} > 30000)$$

PHF – Example

PAY₁

TITLE	SAL
Mech. Eng.	27000
Programmer	24000

PAY₂

TITLE	SAL
Elect. Eng.	40000
Syst. Anal.	34000

PHF – Example

■ Fragmentation of relation PROJ

□ Applications:

- Find the **name and budget of projects** given their no.
 - Issued at three sites
- **Access project** information **according to budget**
 - one site accesses ≤ 200000 other accesses > 200000

□ Simple predicates

□ For application (1)

$p_1 : \text{LOC} = \text{"Montreal"}$

$p_2 : \text{LOC} = \text{"New York"}$

$p_3 : \text{LOC} = \text{"Paris"}$

□ For application (2)

$p_4 : \text{BUDGET} \leq 200000$

$p_5 : \text{BUDGET} > 200000$

□ $Pr = Pr' = \{p_1, p_2, p_3, p_4, p_5\}$

PHF – Example

■ Fragmentation of relation PROJ continued

□ Minterm fragments left after elimination

$m_1 : (\text{LOC} = \text{"Montreal"}) \wedge (\text{BUDGET} \leq 200000)$

$m_2 : (\text{LOC} = \text{"Montreal"}) \wedge (\text{BUDGET} > 200000)$

$m_3 : (\text{LOC} = \text{"New York"}) \wedge (\text{BUDGET} \leq 200000)$

$m_4 : (\text{LOC} = \text{"New York"}) \wedge (\text{BUDGET} > 200000)$

$m_5 : (\text{LOC} = \text{"Paris"}) \wedge (\text{BUDGET} \leq 200000)$

$m_6 : (\text{LOC} = \text{"Paris"}) \wedge (\text{BUDGET} > 200000)$

PHF – Example

PROJ₁

PNO	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal

PROJ₃

PNO	PNAME	BUDGET	LOC
P2	Database Develop.	135000	New York

PROJ₄

PNO	PNAME	BUDGET	LOC
P3	CAD/CAM	255000	New York

PROJ₆

PNO	PNAME	BUDGET	LOC
P4	Maintenance	310000	Paris

PHF – Correctness

■ Completeness

- Since Pr' is complete and minimal, the selection predicates are complete

■ Reconstruction

- If relation R is fragmented into $F_R = \{R_1, R_2, \dots, R_r\}$

$$R = \bigcup_{\forall R_i \in F_R} R_i$$

■ Disjointness

- Minterm predicates that form the basis of fragmentation should be mutually exclusive.

Fragmentation Alternatives – Derived

Derived Horizontal Fragmentation (DHF)

Definition

- Relation fragmented **based on another relation's fragmentation** via a join condition.

Example:

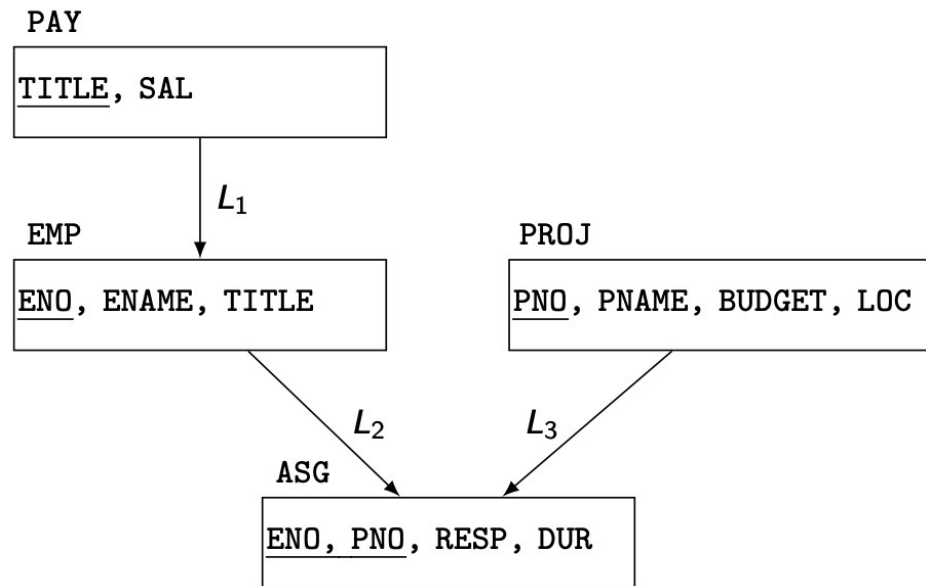
- Suppose **EMP(ENO, ENAME, PNO)** references **PROJ(PNO, BUDGET, LOC)**.
- If PROJ is fragmented by budget:
 - **EMP1** → Employees working on projects in PROJ1
 - **EMP2** → Employees working on projects in PROJ2

Key Point:

- DHF ensures **referential locality** (employees co-located with projects they work on).

DHF – Information Requirements

- Database Information
 - relationship



- cardinality of each relation: $card(R)$ - number of rows in the relation.

DHF – Definition

Given a link L where $owner(L)=S$ and $member(L)=R$, the derived horizontal fragments of R are defined as

$$R_i = R \bowtie_F S_i, 1 \leq i \leq w$$

where w is the maximum number of fragments that will be defined on R and it returns all tuples from R that have a corresponding tuple in fragment S_i when joined using condition F .

$$S_i = \sigma_{F_i}(S)$$

where F_i is the formula according to which the primary horizontal fragment S_i is defined.

DHF – Example

Given link L_1 where $\text{owner}(L_1)=\text{PAY}$ and $\text{member}(L_1)=\text{ASG}$

$$\text{ASG}_1 = \text{ASG} \bowtie \text{PAY}_1$$

$$\text{ASG}_2 = \text{ASG} \bowtie \text{PAY}_2$$

where

$$\text{PAY}_1 = \sigma_{\text{SAL} \leq 30000}(\text{PAY})$$

$$\text{PAY}_2 = \sigma_{\text{SAL} > 30000}(\text{PAY})$$

ASG₁

ENO	PNO	RESP	DUR
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E7	P3	Engineer	36

ASG₂

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E5	P2	Manager	24
E6	P4	Manager	48
E8	P3	Manager	40

Vertical Fragmentation

- More difficult than horizontal, because more alternatives exist.

Two approaches :

- grouping
 - attributes to fragments
- splitting
 - relation to fragments

Fragmentation Alternatives – Vertical

Definition: Splits relation **R** into fragments **R1, R2, ..., Rr**.

- **Each fragment** contains:
 - A subset of **non-key attributes**
 - **Primary key** (replicated for reconstruction).

Goal: Place **together attributes frequently accessed together**.

Applications mostly access **one fragment** → improves performance.

Fragmentation Alternatives – Vertical

Heuristic Approaches

■ Grouping

- ❑ Start with each attribute as separate fragment.
- ❑ Gradually **merge fragments** until criteria met.
- ❑ May produce **overlapping fragments**.

■ Splitting (preferred)

- ❑ Start with full relation.
- ❑ Iteratively **split into non-overlapping fragments** based on access patterns.
- ❑ More suitable for **top-down design**.

Fragmentation Alternatives – Vertical

PROJ₁: information about
project budgets

PROJ₂: information about
project names and
locations

PROJ

<u>PNO</u>	PNAME	BUDGET	LOC
P1	Instrumentation	150000	Montreal
P2	Database Develop.	135000	New York
P3	CAD/CAM	250000	New York
P4	Maintenance	310000	Paris

PROJ₁

PNO	BUDGET
P1	150000
P2	135000
P3	250000
P4	310000

PROJ₂

PNO	PNAME	LOC
P1	Instrumentation	Montreal
P2	Database Develop.	New York
P3	CAD/CAM	New York
P4	Maintenance	Paris

Vertical Fragmentation

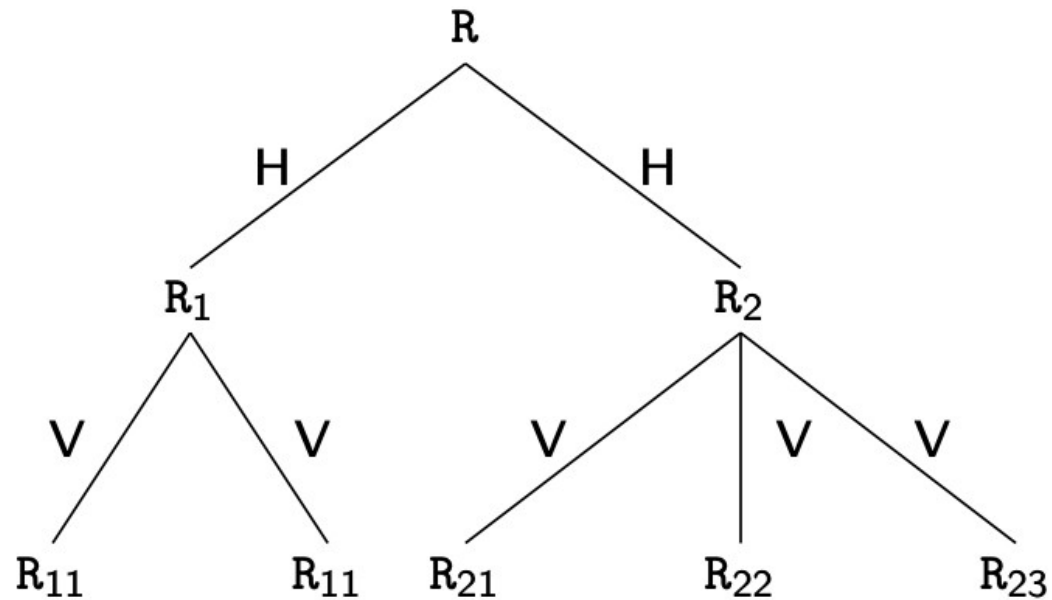
- Overlapping fragments
 - ▣ grouping
- Non-overlapping fragments
 - ▣ splitting

We do not consider the replicated key attributes to be overlapping.

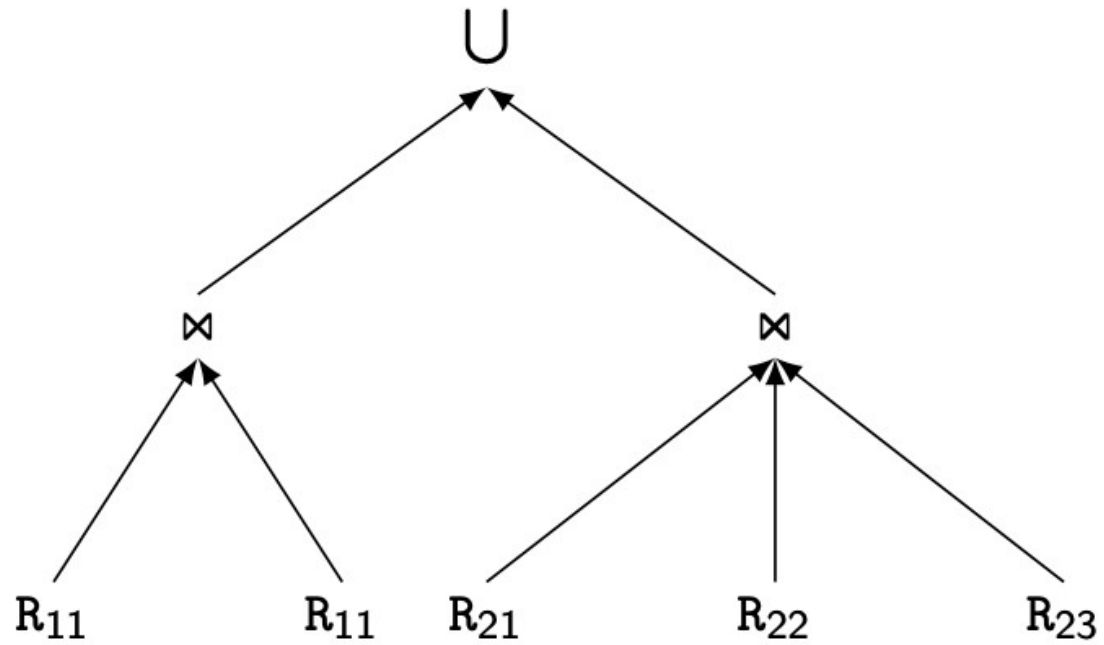
Advantage:

Easier to enforce functional dependencies
(for integrity checking etc.)

Hybrid Fragmentation



Reconstruction of HF



Outline

- Distributed and Parallel Database Design
 - ▣ Fragmentation
 - ▣ Data distribution
 - ▣ Combined approaches

Allocation Alternatives

- Non-replicated

- partitioned : each fragment resides at only one site

- Replicated

- fully replicated : each fragment at each site
 - partially replicated : each fragment at some of the sites

Comparison of Replication Alternatives

	Full replication	Partial replication	Partitioning
QUERY PROCESSING	Easy	Same difficulty	
DIRECTORY MANAGEMENT	Easy or nonexistent	Same difficulty	
CONCURRENCY CONTROL	Moderate	Difficult	Easy
RELIABILITY	Very high	High	Low
REALITY	Possible application	Realistic	Possible application

Outline

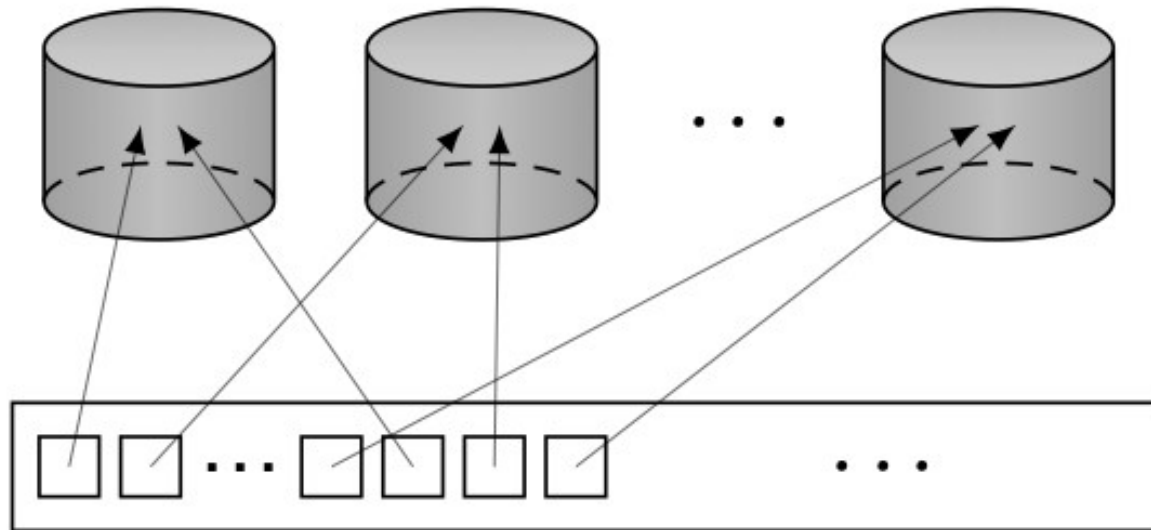
- Distributed and Parallel Database Design
 - Fragmentation
 - Data distribution
 - Combined approaches

Combining Fragmentation & Allocation

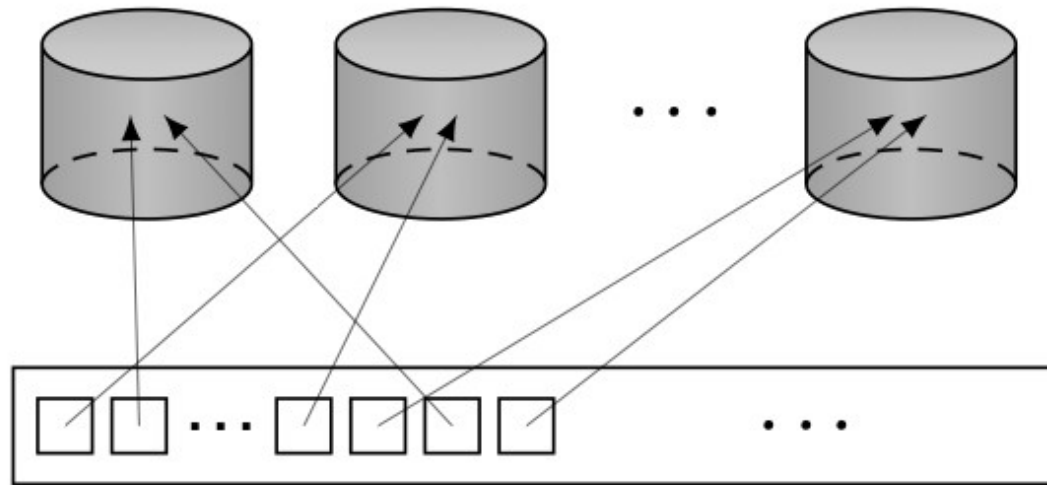
Partition the data to dictate where it is located

- Workload-agnostic techniques
 - Round-robin partitioning
 - Hash partitioning
 - Range partitioning
- Workload-aware techniques
 - Graph-based approach

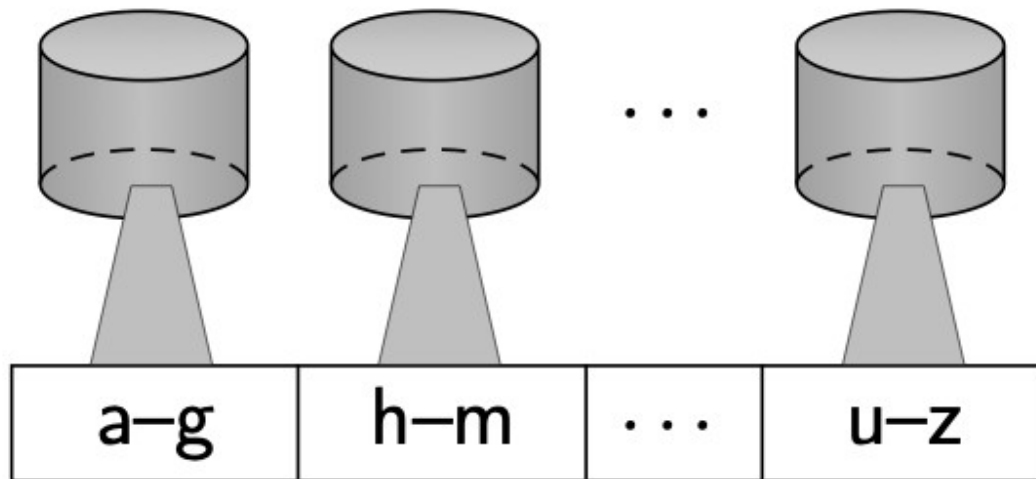
Round-robin Partitioning



Hash Partitioning



Range Partitioning

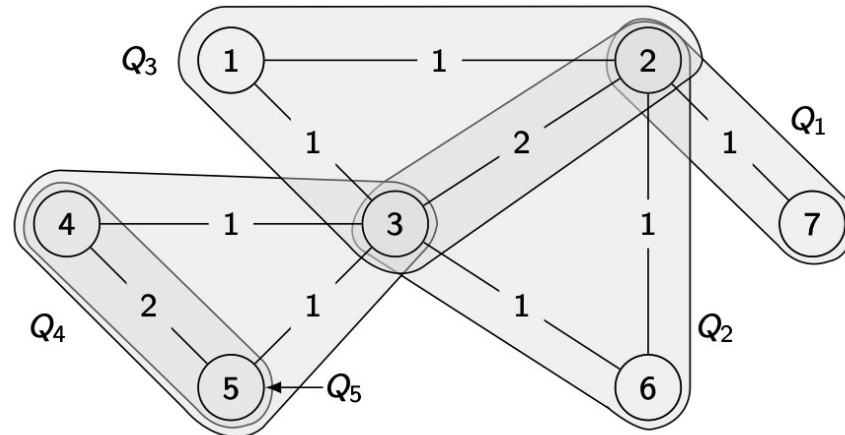


Workload-Aware Partitioning

■ Example:

Graph $G=(V,E)$ where

- vertex $v_i \in V$ represents a tuple in database,
 - edge $e=(v_i, v_j) \in E$ represents a query that accesses both tuples v_i and v_j ;
 - each edge has weight counting the no. of queries that access both tuples
- Perform vertex disjoint graph partitioning
- Each vertex is assigned to a separate partition



Incorporating Replication

- Replicate each vertex based on the no. of transactions accessing that tuple → each transaction accesses a separate copy

